Universidade Federal do ABC
MCTA028-15 Programação Estruturada, 2026.3
Profs. Roberto Sadao Yokoyama e Luneque Del Rio de Souza e Silva Jr.

Relatório Técnico

projetoPE: compactador e descompactador de arquivos com o algoritmo de Huffman

Integrantes

José Ricardo Feltran Júnior, RA 11202421133
Rafael Maximiano Luiz, RA 11202432420
Renan Henrique Ferreira Monteiro, RA 11202020303
Rodrigo Martel Poggi de Araújo, RA 11202522393

Santo André, novembro de 2026

1. Requisitos implementados e funcionais

O projetoPE é um programa em C que comprime e descomprime qualquer tipo de arquivo (texto ou binário) usando codificação de Huffman por byte. Tudo que o enunciado pede está implementado e foi testado:

- **Menu no terminal** com Comprimir, Descomprimir e Sair, mais três opções extras: ver a tabela de códigos de Huffman de um arquivo, testar um arquivo (comprime, descomprime e compara com o original) e informações da equipe.
- **Caminhos relativos e absolutos.** O programa também tira aspas e espaços do começo e do fim, o que acontece quando o arquivo é arrastado para o terminal.
- **Extensão própria .abcz** na compressão. Na descompressão o usuário informa o nome do arquivo restaurado (se apertar ENTER, o programa usa o nome sem o .abcz).
- **Preservação dos originais.** O arquivo de entrada só é aberto para leitura e cada operação gera um arquivo novo. Se entrada e saída forem o mesmo caminho, a operação é recusada. Em caso de erro, o arquivo de saída incompleto é apagado.
- **Métricas:** tamanho original e comprimido em bytes, razão de compressão R = Sc / So e tempo de processamento.
- **Funcionalidade adicional:** verificação de integridade com CRC-32. O checksum do arquivo original é gravado no cabeçalho e recalculado na descompressão.

2. Fundamentação conceitual

2.1 Codificação de Huffman

Num arquivo comum, cada byte ocupa 8 bits, mesmo que alguns valores apareçam muito mais que outros. O algoritmo de Huffman (1952) dá códigos curtos para os bytes mais frequentes e códigos longos para os raros. Os códigos são prefixados: nenhum código é o começo de outro, então dá para decodificar a sequência de bits sem separadores. Para um modelo que olha cada byte de forma independente, Huffman gera o menor tamanho médio possível.

Seguimos as etapas do enunciado: (1) ler o arquivo e contar a frequência de cada um dos 256 valores de byte; (2) montar a árvore; (3) gerar os códigos; (4) ler o arquivo de novo trocando cada byte pelo seu código e gravar o cabeçalho; (5) na descompressão, ler o cabeçalho, remontar a mesma árvore e percorrê-la bit a bit.

2.2 Fila de prioridade (min-heap)

Para montar a árvore é preciso, várias vezes, tirar os dois nós de menor frequência. Usamos um min-heap num vetor, em que inserir e remover o mínimo custam O(log n). Com no máximo 256 folhas, a árvore inteira sai em O(n log n). Cada folha é um byte presente no arquivo. A cada passo os dois menores viram filhos de um nó novo, cuja frequência é a soma das duas, até sobrar só a raiz.

Um detalhe importante: a descompressão precisa montar exatamente a mesma árvore. Quando duas frequências empatam, a ordem de saída do heap poderia variar. Por isso cada nó tem um número de criação (campo *ordem*) e o empate é decidido por ele. Assim a mesma tabela de frequências sempre gera a mesma árvore.

2.3 CRC-32

Para a verificação de integridade usamos o CRC-32 (polinômio 0xEDB88320, o mesmo do gzip e do ZIP), calculado com uma tabela de 256 entradas, em tempo linear. Ele detecta qualquer erro de 1 bit e quase qualquer outra alteração, o que a soma simples dos bytes não garante.

3. Bibliotecas, decisões de projeto e implementação

3.1 Bibliotecas

Só a biblioteca padrão do C: `stdio.h` (arquivos e terminal), `stdlib.h` (malloc/free, strtol), `string.h` (memset, memcpy, strlen e strcmp, usados só no menu e para zerar estruturas, nunca no processamento da compressão), `time.h` (clock, para medir o tempo), `stdint.h` e `inttypes.h` (inteiros de tamanho fixo para o cabeçalho) e `ctype.h`. Não usamos nenhuma biblioteca de compressão, nem qsort: o heap, a árvore, a leitura e escrita de bits e o CRC-32 foram escritos por nós. O código compila com `gcc -Wall -Wextra` sem nenhum aviso.

3.2 Organização dos módulos

Módulo	Responsabilidade
heap.c / heap.h	min-heap de ponteiros para nós da árvore
huffman.c / huffman.h	contagem de frequências, árvore e tabela de códigos
bitio.c / bitio.h	BitWriter e BitReader (gravam e leem bit a bit)
cabecalho.c / cabecalho.h	formato do cabeçalho .abcz
checksum.c / checksum.h	CRC-32
compressao.c / descompressao.c	rotinas completas de comprimir e descomprimir
main.c	menu, leitura de caminhos e exibição das métricas

3.3 Heap e árvore

A comparação do heap usa a frequência e, no empate, a ordem de criação do nó:

```
static int menor(const HuffNode *a, const HuffNode *b) {
    if (a->freq != b->freq)
```

```
        return a->freq < b->freq;
    return a->ordem < b->ordem;
}
```

Construção da árvore, juntando os dois menores até sobrar um. Se um malloc falhar, tudo que já foi alocado é liberado:

```
/* junta os 2 menores ate sobrar 1 */
while (heap_tamanho(h) > 1) {
    a = heap_remover_min(h);
    b = heap_remover_min(h);
    pai = novo_no(0, a->freq + b->freq, prox_ordem++, a, b);
    if (pai == NULL) {
        huff_liberar_arvore(a);
        huff_liberar_arvore(b);
        liberar_heap_com_nos(h);
        return NULL;
    }
    heap_inserir(h, pai);
}
```

Dois casos especiais: arquivo vazio (não há árvore, o .abcz fica só com o cabeçalho) e arquivo com um único valor de byte (a raiz recebe essa folha à esquerda, então o código dele é "0", com 1 bit).

3.4 Escrita de bits

Os códigos têm tamanhos que não são múltiplos de 8. O BitWriter acumula os bits num byte e só grava no arquivo quando junta 8 (bit mais significativo primeiro). No final, o último byte é completado com zeros.

```
int bw_put_bit(BitWriter *bw, int bit) {
    bw->buffer = (uint8_t)((bw->buffer << 1) | (bit & 1));
    bw->nbits++;
    if (bw->nbits == 8) {
        if (fputc(bw->buffer, bw->f) == EOF)
            return -1;
        bw->buffer = 0;
        bw->nbits = 0;
    }
    return 0;
}
```

3.5 Formato do arquivo .abcz

Campo	Tamanho	Conteúdo
Assinatura	4 bytes	"ABCZ"
Versão	1 byte	1
Tamanho original	8 bytes	número de bytes do arquivo original
Checksum	4 bytes	CRC-32 do arquivo original
Quantidade de símbolos	2 bytes	quantos bytes distintos aparecem
Frequências	variável	para cada símbolo: o byte e a frequência em varint
Dados	variável	bits dos códigos, completados com zeros no fim

Em vez de gravar as 256 frequências com 8 bytes cada (2 KB fixos), gravamos só os bytes que aparecem, e cada frequência em varint (7 bits por byte, o oitavo indica se continua). Frequências pequenas ocupam 1 byte. Por isso um arquivo vazio vira um .abcz de 19 bytes. A leitura do

cabeçalho confere se a soma das frequências bate com o tamanho original, o que barra arquivos inválidos antes de decodificar.

3.6 Compressão e descompressão

A compressão lê o arquivo duas vezes, em blocos de 64 KB, então o arquivo nunca é carregado inteiro na memória. Na 1ª passada conta as frequências e calcula o CRC-32. Depois monta a árvore e a tabela de códigos (uma vez só, sem procurar o código na árvore para cada byte). Na 2ª passada grava os bits. A descompressão percorre a árvore a partir da raiz (bit 0 vai para a esquerda, 1 para a direita) até chegar numa folha, e para quando completa o tamanho original, ignorando os zeros de preenchimento:

```
for (i = 0; i < cab.tam_original; i++) {
    no = raiz;
    while (!huff_eh_folha(no)) {
        bit = br_get_bit(&br);
        if (bit == EOF) { ret = DECOMP_ERRO_LEITURA; goto fim; }
        no = bit ? no->dir : no->esq;
        if (no == NULL) { ret = DECOMP_ERRO_FORMATO; goto fim; }
    }
    bloco[pos++] = no->byte;
    if (pos == TAM_BLOCO) {
        crc = checksum_crc32(crc, bloco, pos);
        if (fwrite(bloco, 1, pos, sai) != pos) { ret = DECOMP_ERRO_ESCRITA; goto fim; }
        pos = 0;
    }
}
```

Memória: os buffers de 64 KB são estáticos, a árvore tem no máximo 511 nós e o heap é alocado uma vez. Rodamos o programa no Valgrind (compressão, descompressão e o menu) sem nenhum vazamento ou acesso inválido.

4. Casos de teste

Além dos testes abaixo, comprimimos e descomprimimos arquivos de 1 byte, 5000 bytes iguais, 300 KB aleatórios e 4 MB de texto. Todos voltaram idênticos ao original. Também testamos arquivos inexistentes, truncados e que não são .abcz: o programa mostra a mensagem de erro e volta ao menu.

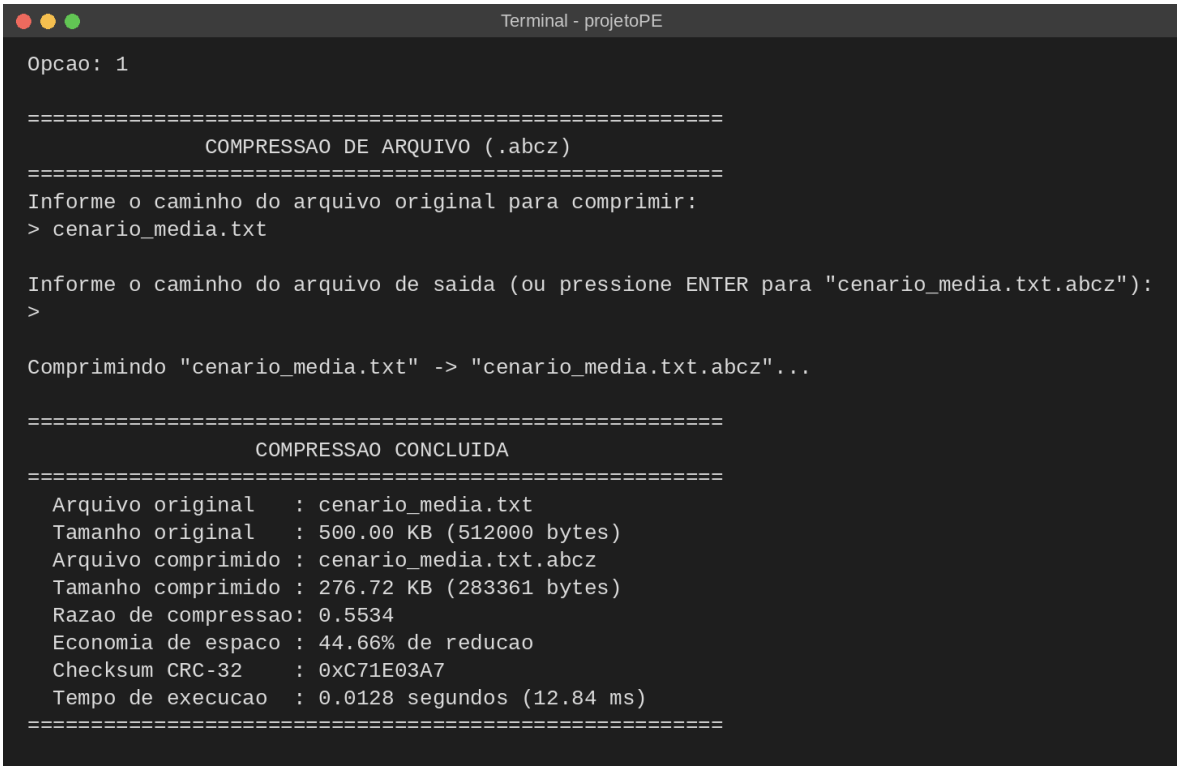


Figura 1: compressão do arquivo do cenário de média redundância (500 KB).

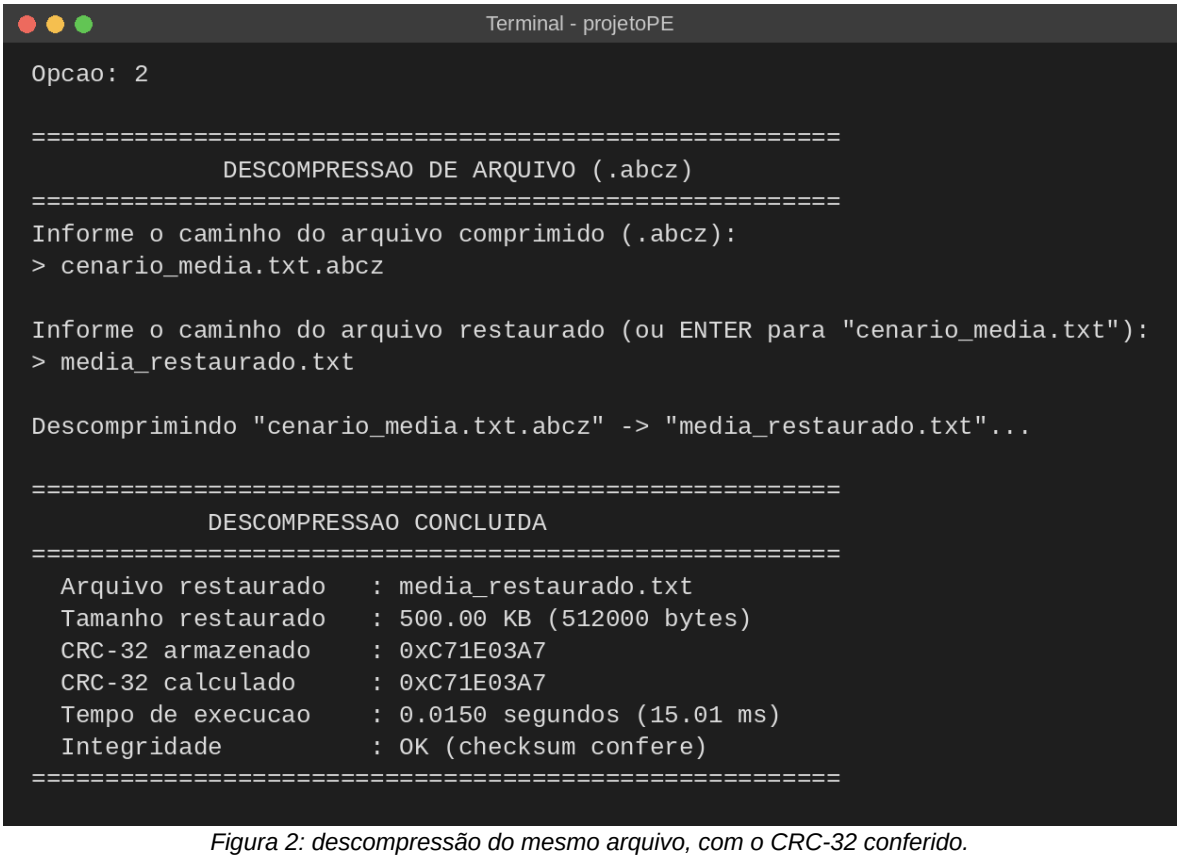


Figura 2: descompressão do mesmo arquivo, com o CRC-32 conferido.

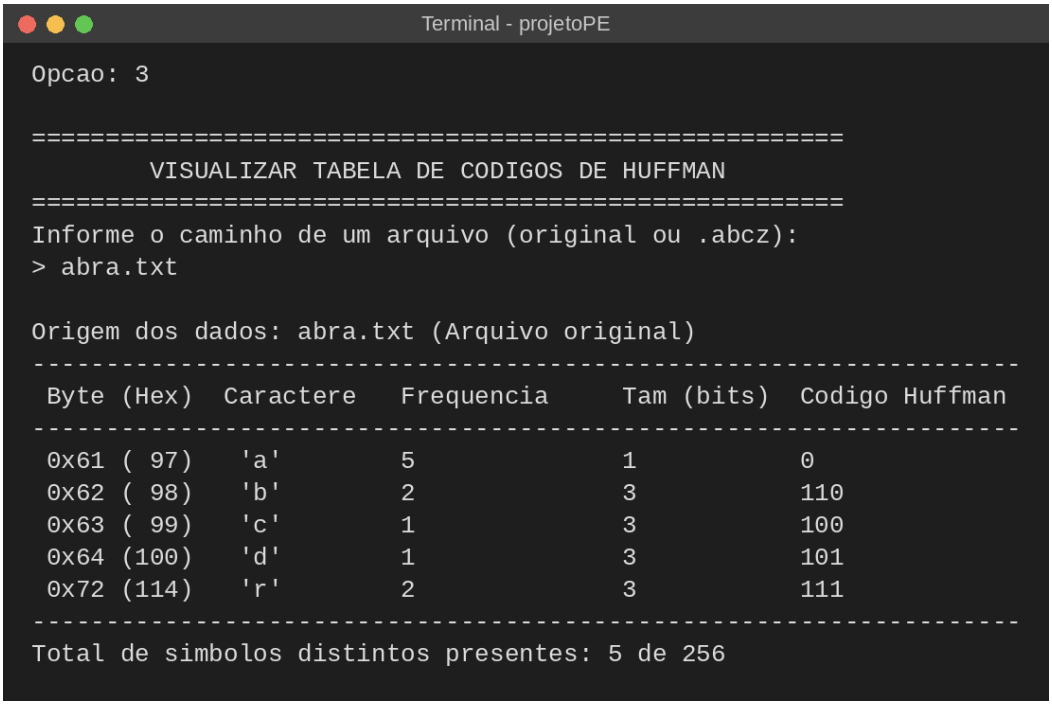


Figura 3: tabela de códigos de Huffman do arquivo "abracadabra" (11 bytes).

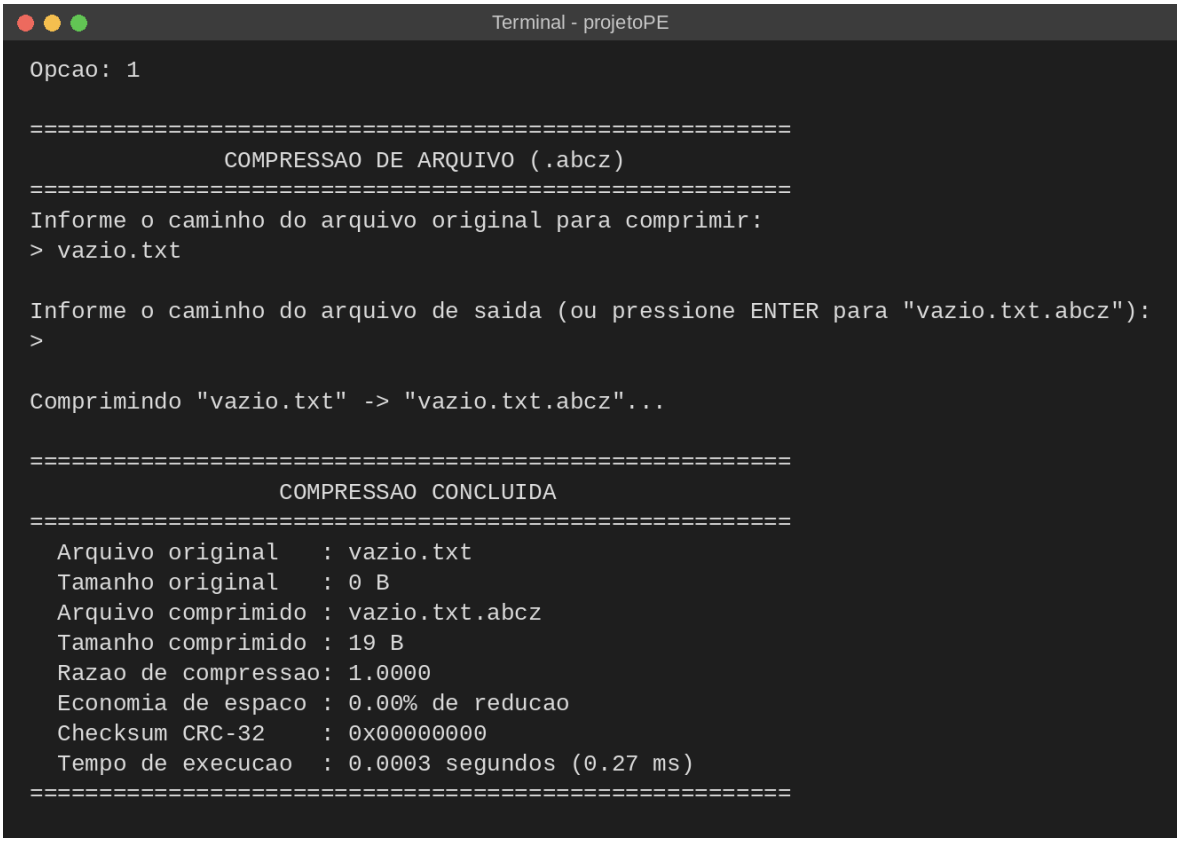


Figura 4: arquivo vazio. O .abcz fica só com o cabeçalho (19 bytes).

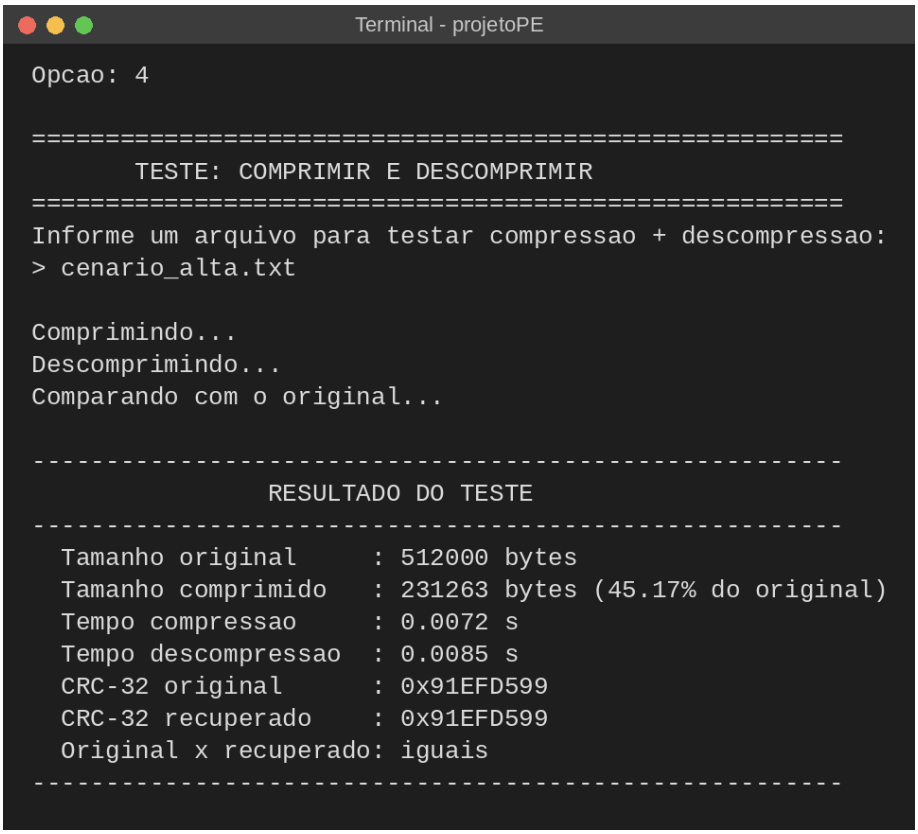


Figura 5: opção 4, que comprime, descomprime e compara byte a byte (cenário de alta redundância).

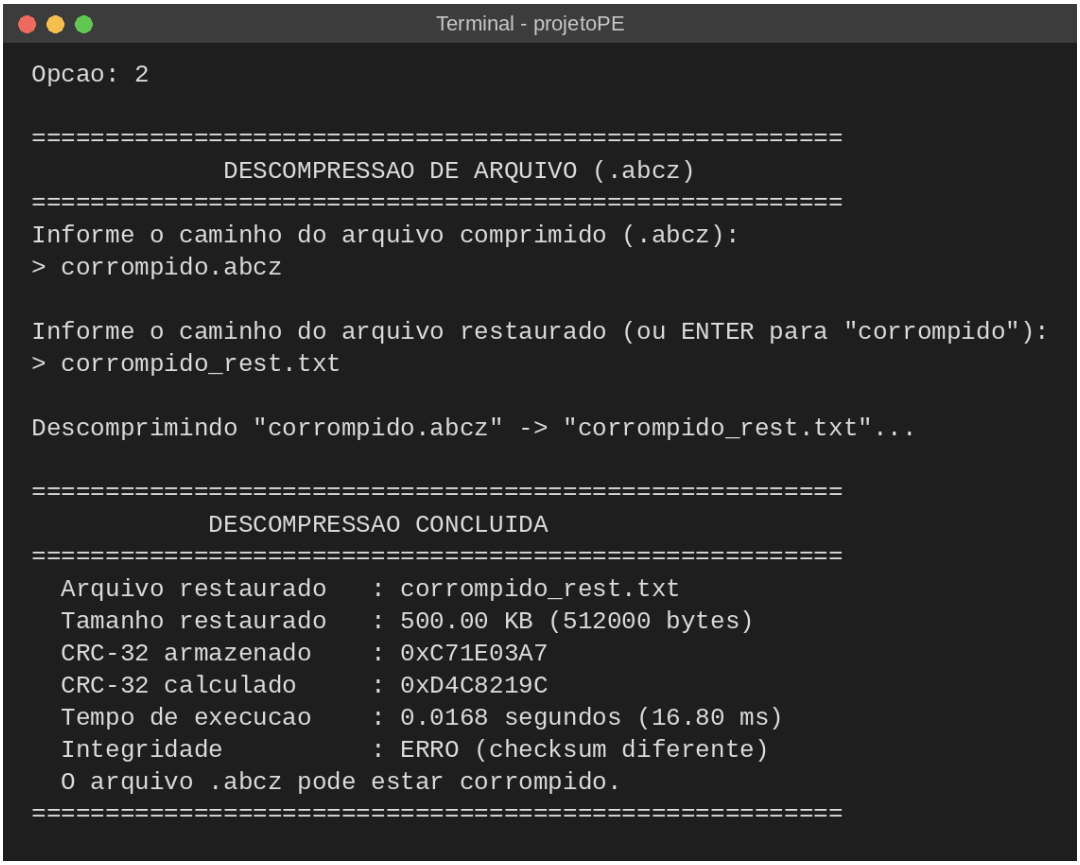


Figura 6: .abcz com um bit alterado. O checksum calculado não bate com o armazenado.

5. Desempenho e análise dos resultados

Usamos o script `tests/benchmark_gzip.py`, que gera três arquivos de 512.000 bytes e compara o nosso programa com o gzip do Linux (nível padrão). Alta redundância: um trecho de 49 caracteres ("ATGCATGC..." seguido de um texto fixo) repetido até completar o arquivo. Média redundância: frases sobre a disciplina sorteadas de uma lista pequena. Baixa redundância: bytes aleatórios. Incluímos também um texto real, o próprio código-fonte do projeto (44.251 bytes). Em todos os casos os arquivos restaurados tiveram o mesmo SHA-256 do original.

Cenário	Original (B)	Huffman (B)	R Huffman	gzip (B)	R gzip
Alta redundância	512.000	231.263	0,452	1.578	0,003
Média redundância	512.000	283.361	0,553	6.588	0,013
Baixa redundância	512.000	512.787	1,002	512.116	1,000
Texto real (código-fonte)	44.251	28.004	0,633	11.047	0,250

Tabela 1: tamanhos e razão de compressão R = Sc / So.

Cenário	Comp. Huffman	Comp. gzip	Descomp. Huffman	Descomp. gzip
Alta redundância	8,7 ms	4,2 ms	10,5 ms	3,5 ms
Média redundância	13,6 ms	4,5 ms	16,8 ms	3,4 ms
Baixa redundância	14,4 ms	18,9 ms	18,8 ms	4,7 ms

Tabela 2: tempo médio de 3 execuções (Linux, gcc -O2), incluindo a abertura do processo.

Onde ficamos perto do gzip. Em dados aleatórios os dois praticamente empatam: nenhum compressor sem perdas consegue reduzir dados sem padrão, e os dois só aumentam um pouco por causa do cabeçalho (o nosso cresce 787 bytes, 0,15%, por causa das 256 frequências). Nesse caso o nosso programa comprimiu até mais rápido que o gzip, porque o gzip perde tempo procurando repetições que não existem.

Onde ficamos longe. Nos arquivos com repetição o gzip é muito melhor (R = 0,003 contra 0,452 no cenário de alta). O gzip usa o Deflate, que combina LZ77 com Huffman: o LZ77 troca trechos repetidos por uma referência (distância, comprimento) para algo que já apareceu antes, e só depois aplica Huffman. O nosso programa só olha a frequência de cada byte isolado, sem considerar a ordem. No cenário de alta, o gzip percebe que o mesmo trecho de 49 caracteres se repete e guarda praticamente uma cópia só. Para o Huffman, o arquivo é só uma mistura de cerca de 20 caracteres, e o melhor que ele faz é usar uns 3,6 bits por caractere (R = 0,45). Os cenários gerados pelo script são bem mais repetitivos que um texto comum, o que exagera a vantagem do gzip. No texto real a diferença é menor (0,633 contra 0,250), mas ainda grande pelo mesmo motivo.

Tempo. O nosso programa leva entre 9 e 19 ms para 500 KB, nas duas direções. A descompressão é um pouco mais lenta que a compressão porque percorre a árvore bit a bit, enquanto a compressão consulta a tabela de códigos direto. O gzip é mais otimizado e descomprime bem mais rápido. Os tempos dependem da máquina; na apresentação vamos medir

de novo no laboratório.

6. Participação dos integrantes

- **José Ricardo Feltran Júnior:** min-heap (heap.c), árvore de Huffman com o desempate pela ordem de criação e geração dos códigos (huffman.c).
- **Rafael Maximiano Luiz:** escrita e leitura de bits (bitio.c), formato do cabeçalho com varint (cabecalho.c) e rotina de compressão em duas passadas (compressao.c).
- **Renan Henrique Ferreira Monteiro:** rotina de descompressão (descompressao.c) e verificação de integridade com CRC-32 (checksum.c).
- **Rodrigo Martel Poggi de Araújo:** menu e tratamento de caminhos (main.c), medição de tempo e tamanho, testes comparativos com o gzip, leia-me.txt, Makefile e relatório.

A integração e os testes finais foram feitos pelo grupo todo.